Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune

Implementation Solution Guide

Version 1.0.0 · 2026-06-27

Contents

Executive Summary — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune 5

What you get . . . 5

Why it matters . . . 5

Approach . . . 5

1. Solution Overview — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune 6

What this solution delivers . . . 6

Devices in scope . . . 6

What is in scope vs. referenced only . . . 7

Critical accuracy note — get the Epic specifics from your Epic build team . . . 7

Why this approach . . . 7

How to use this package . . . 7

2. Requirements Guide — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune 8

R1. Microsoft Intune licensing and app-management rights . . . 8

R2. Microsoft Win32 Content Prep Tool . . . 8

R3. The Epic installer — obtained from Epic Technical Services / UserWeb . . . 8

R4. Target operating system — Windows 10 / Windows 11 . . . 9

R5. WebView2 runtime (Hyperdrive prerequisite) . . . 9

R6. Configuration Manager co-management (only if applicable) . . . 9

R7. Entra device groups for deployment rings . . . 9

R8. Detection / version strategy . . . 10

R9. A test clinical device (not in patient care) . . . 10

R10. Approved clinical change window and rollback plan . . . 10

R11. Imprivata present (only if integrating SSO / badge-tap) . . . 10

Requirements checklist (use before packaging) . . . 10

3. Implementation Guide — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune 11

Phase A — Assess . . . 11

Phase B — Package . . . 12

Phase C — Pilot . . . 13

Phase D — Validate . . . 14

Phase E — Broad Deploy . . . 14

Phase F — Lifecycle (update / supersede / retire) . . . 15

5. Testing Guide — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune 16
Test 1 — The Epic client installed . . . 16
Test 2 — The installed version is the target version . . . 16
Test 3 — Intune reports the app Installed (no errors) . . . 16
Test 4 — The client launches and loads . . . 16
Test 5 — WebView2 runtime present (Hyperdrive) . . . 16
Test 6 — Imprivata badge-tap into Epic still works (if integrating) . . . 16
Test 7 — No leftover old version after an upgrade / retire . . . 17
Test 8 — Ring rollout is healthy . . . 17
Test 9 — Device compliance and clinical function intact . . . 17
Acceptance gate . . . 17
6. Troubleshooting Guide — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune 17
A. Install fails or never starts . . . 18
B. Detection mismatch (installs but Intune says "Not installed", or re-installs in a loop) . . . 18
C. Version drift (devices stuck on an old version) . . . 19
D. Hyperdrive won't launch (blank/white window, crash) — WebView2 . . . 19
E. Supersedence not upgrading (old version stays) . . . 19
F. Retire leftovers (old client remains after retire) . . . 20
G. Ring / assignment problems . . . 20
H. Imprivata badge-tap no longer launches Epic after a client change . . . 20
Escalation / rollback . . . 21
7. Best Practices — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune 21
Source of truth: Epic owns the installer, you own the delivery . . . 21
Pin the version with detection — and keep detection honest . . . 21
Always pilot before broad — and let supersedence respect rings . . . 21
Treat the WebView2 runtime as a first-class dependency (Hyperdrive) . . . 22
Upgrade cleanly with supersedence; retire deliberately . . . 22
Respect the clinical environment . . . 22
Co-management clarity . . . 22
Protect ePHI in operations . . . 22
Operate on a cadence . . . 22
8. Deliverables Summary — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune 23
Documentation . . . 23
PowerShell toolkit (`scripts\`) . . . 24

Visual asset 24
Named objects you create (reused verbatim across the docs and scripts) 24
Appendix A — PowerShell Scripts 25

Executive Summary — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune

The Epic clinical EHR client is the most important application on a clinical endpoint — and one of the hardest to manage well at scale. It runs on shared, always-on devices used directly in patient care (clinical workstations, workstations-on-wheels, exam-room PCs), it must stay version-coherent with the organization's Epic environment, and a bad client build pushed to the floor is a patient-safety event, not just an IT incident. This solution turns Epic client deployment from a manual, error-prone task into a **repeatable, ring-based, fully auditable lifecycle** managed through **Microsoft Intune** (and Microsoft Configuration Manager where devices are co-managed).

What you get

- **A productized packaging and deployment method** for the Epic client — both the current **Hyperdrive** (Chromium / WebView2-based) and legacy **Hyperspace** — as an Intune **Win32 app** with **version-pinned detection**, requirement rules, and (for Hyperdrive) a **WebView2 runtime dependency**.
- **Ring-based rollout** using named Entra device groups (`Epic Hyperdrive – Pilot Ring`, `Epic Hyperdrive – Broad Ring`, `EPIC – Clinical Workstations`) so every release and upgrade is proven on a small, representative set of devices before reaching the clinical floor.
- **A validation gate** — documented tests plus PowerShell tooling — confirming the correct version installed, the client launches and loads, and **Imprivata badge-tap into Epic still works** before the rollout is allowed to expand.
- **A clean lifecycle** — upgrades delivered via Intune **supersedence** (the new version automatically uninstalls and replaces the old), and a deliberate **retire / uninstall** path so no stale Epic client is ever left behind on a shared device.
- **Five operational PowerShell tools** for discovery, validation, fleet reporting, safe remediation, and ring-group provisioning.

Why it matters

- **Patient-safety containment:** ring-based rollout limits the blast radius of any defective client build to a handful of validated, non-care devices.
- **Version coherence with Epic:** version pinning and supersedence keep the entire fleet aligned to the exact client version the Epic environment expects, eliminating drift and coexisting-version problems on shared devices.
- **Auditable, compliant change:** every deploy, upgrade, and retire is change-controlled, windowed, reversible, and recorded — aligned to HIPAA / HITRUST expectations for a clinical setting.

Approach

The work follows the **standard deployment model**: **Assess** → **Package** → **Pilot** → **Validate** → **Broad Deploy** → **Lifecycle (update / supersede / retire)**. Epic-specific installer, version, and configuration values are obtained from the organization's **Epic build team / Epic UserWeb** and substituted into an otherwise exact, vendor-accurate Intune / ConfigMgr packaging and lifecycle method — so the deployment mechanics are correct and the Epic specifics are always authoritative.

1. Solution Overview — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune

This solution gives you a repeatable, auditable way to **package, deploy, validate, update, and retire the Epic clinical EHR client** across a clinical endpoint fleet using **Microsoft Intune** (and Microsoft Configuration Manager / SCCM where devices are co-managed). It covers both Epic's current **Hyperdrive** client (the Chromium / WebView2-based client) and the legacy **Hyperspace** client, and it treats the EHR client as a managed, versioned application with a defined lifecycle — not a one-time install.

The work follows the **standard deployment model**: **Assess** → **Package** → **Pilot** → **Validate** → **Broad Deploy** → **Lifecycle (update / supersede / retire)**.

What this solution delivers

• A **Win32 app package** (`.intunewin`) of the Epic client, built with the Microsoft Win32 Content Prep Tool, with **named detection, requirement, and (for upgrades) supersedence rules** that pin the deployment to the exact Epic version your environment runs.

• **Deployment rings** as named Microsoft Entra device groups — `EPIC – Clinical Workstations` (the population), `Epic Hyperdrive – Pilot Ring`, and `Epic Hyperdrive – Broad Ring` — so every release is proven on a small, representative set of clinical endpoints before it reaches the floor.

• A **validation gate** (documented tests plus PowerShell scripts) that confirms the correct version installed, the client launches and loads, the WebView2 runtime is present (Hyperdrive), and the Imprivata badge-tap single sign-on still works before the rollout is allowed to expand.

• A **lifecycle process**: upgrades delivered via Intune **supersedence** (new version automatically replaces and uninstalls the old one), and a clean **retire / uninstall** path for decommissioned versions so no stale Epic client is left behind.

• Five PowerShell tools (discovery, validation, fleet reporting, safe remediation, and ring-group provisioning) to operate the deployment day to day.

Devices in scope

• **Clinical workstations** (fixed exam-room and nursing-station PCs).

• **Workstations-on-wheels (WOWs)** / mobile carts.

• **Exam-room PCs** and other shared clinical endpoints.

These are typically **shared, multi-user, always-on** devices on a clinical network, which is why ring-based rollout, tight version pinning, and validated update windows matter more here than on a standard office fleet.

What is in scope vs. referenced only

In scope (this solution): Intune / ConfigMgr **packaging, detection, requirement and supersedence rules, ring assignment, validation, and the update / retire lifecycle** of the Epic client.

Referenced only (covered in depth by separate solutions):

- **Imprivata** single sign-on / badge-tap / EPCS configuration on clinical endpoints — this solution only **validates that tap-to-launch into Epic still works** after a client change.
- **Clinical peripherals** (signature pads, label/wristband printers, document scanners, card readers) — referenced as validation touchpoints only.

Critical accuracy note — get the Epic specifics from your Epic build team

Epic's installer package, command-line switches, the exact **client version string**, and the **environment / connection configuration** (which Epic environment the client points to, e.g. PRD) are defined in **Epic's customer-only documentation (UserWeb / Galaxy)** and by **your organization's Epic Technical Services / client systems team**. This solution **does not invent** Epic-internal installer flags or version numbers — and you should not either.

Throughout the Implementation Guide you will see placeholders such as , , , and . **Obtain the real values from your Epic build team / Epic UserWeb** and substitute them consistently. The Intune / ConfigMgr mechanics around those values — packaging, detection, rings, supersedence, retire — are documented here exactly and accurately.

Why this approach

- **Patient-safety / clinical continuity:** the EHR client is mission-critical. Ring-based rollout contains the blast radius of a bad client build to a handful of validated devices, not the whole floor.
- **Version coherence with Epic:** Epic client versions are tied to your Epic environment's upgrade cadence. Pinning the deployed version (via detection) and upgrading via supersedence keeps the fleet aligned to the version Epic expects you to run.
- **Clean lifecycle:** supersedence and a defined retire path prevent the classic problem of multiple Epic client versions accreting on shared clinical devices.

How to use this package

Read the documents in order:

1. **02 – Requirements** and **global-requirements-healthcare-solutions** — confirm everything is in place before you build anything.
1. **03 – Implementation** — package, pilot, validate, broad-deploy, then run the lifecycle.

2. **05 – Testing** — the acceptance gate that must pass before expansion.
3. **07 – Best Practices**, **08 – Deliverables Summary**, and **06 – Troubleshooting** as needed.

2. Requirements Guide — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune

Verify **every** requirement below before you package or deploy anything. Each entry follows the same pattern: **Description · Why required · How to verify · How to implement if missing.** Do **not** assume any prerequisite already exists.

> *Category-wide prerequisites (a tenant, Intune subscription, device enrollment, HIPAA / HITRUST change control, clinical maintenance windows, a test device, rollback posture, etc.) are documented once in* `global-requirements-healthcare-solutions.md`*. This guide lists what is* ***specific to*** *deploying the Epic client.*

R1. Microsoft Intune licensing and app-management rights

Description: An Intune subscription and an operator with rights to create and assign **Windows apps** (Intune Administrator or Application Manager). **Why required:** The Epic client is delivered as an Intune **Win32 app**; without app-management rights you cannot create the package, detection rules, or assignments. **How to verify:** `https://intune.microsoft.com` → **Apps** → **Windows** loads and **+ Add** is available; Entra → **Roles and administrators** shows the operator's role. **How to implement if missing:** Assign an Intune-bearing license and the **Intune Administrator** (or **Application Manager**) role to the operator. See global requirement G1.

R2. Microsoft Win32 Content Prep Tool

Description: The **Microsoft Win32 Content Prep Tool** (`IntuneWinAppUtil.exe`), used to wrap the Epic installer and its files into a single `.intunewin` package. **Why required:** Intune Win32 apps are deployed only as `.intunewin` packages; this is the only supported way to wrap them. **How to verify:** `IntuneWinAppUtil.exe -v` runs from your packaging workstation. **How to implement if missing:** Download the Win32 Content Prep Tool from Microsoft's official GitHub repository (`microsoft/Microsoft-Win32-Content-Prep-Tool`) onto a clean packaging workstation. Keep it current.

R3. The Epic installer — obtained from Epic Technical Services / UserWeb

Description: The exact **Epic client installer** (Hyperdrive and/or Hyperspace), its **silent install / uninstall command lines**, the target **client version string**, the **install directory**, the **launch executable**, and the **environment / connection configuration** for your Epic environment. **Why required:** These values drive the install command, the version-pinned detection rule, the requirement rule, and the supersedence relationship. They are **Epic-specific and customer-only** — they are not documented here and must not be guessed. **How to verify:** Confirm with your **Epic build / Technical Services team** that you have the correct installer and switches for your environment, sourced from **Epic UserWeb / Galaxy**. Run the

installer's silent command on the test device once, by hand, to confirm it installs cleanly and unattended. **How to implement if missing:** Request the installer package, silent switches, version string, and environment configuration from your Epic build team / Epic UserWeb. Record them as the placeholder values , , , , , and used throughout the Implementation Guide.

R4. Target operating system — Windows 10 / Windows 11

Description: Clinical endpoints run a supported, patched **Windows 10 (21H2+)** or **Windows 11**, x64. **Why required:** The Epic client and the WebView2 runtime (for Hyperdrive) require a current, supported Windows build; the Win32 app **requirement rule** will target these OS versions. **How to verify:** `winver` on a device, or Intune → device → **Hardware**. **How to implement if missing:** Patch via Update rings or upgrade devices. See global requirement G2.

R5. WebView2 runtime (Hyperdrive prerequisite)

Description: The **Microsoft Edge WebView2 Runtime** present on every device that will run **Hyperdrive** (Hyperdrive is Chromium / WebView2-based). **Why required:** Hyperdrive will not launch or render without the WebView2 runtime; this is the most common Hyperdrive launch failure. **How to verify:** Run `EPICCLIENT-Validation.ps1` (checks for the WebView2 runtime), or check **Apps & features** / the registry for *Microsoft Edge WebView2 Runtime*. **How to implement if missing:** Deploy the **Evergreen WebView2 Runtime** as its own Intune Win32 / Microsoft Store app and set it as a **dependency** of the Epic Hyperdrive app (Implementation Step 6), so WebView2 installs first. Modern Windows 11 includes WebView2, but **do not assume it** — verify.

R6. Configuration Manager co-management (only if applicable)

Description: If clinical endpoints are **co-managed** with Microsoft Configuration Manager (SCCM), confirm which authority owns **client apps** — Intune or ConfigMgr. **Why required:** With co-management, the **Client apps** workload must be pointed at Intune (or at ConfigMgr) so app assignments take effect from the expected authority and you do not deploy the same client twice. **How to verify:** Intune → **Devices** → **Configuration Manager** → **Co-management**, review the **Client apps** workload slider; or in the ConfigMgr console, review co-management settings. **How to implement if missing:** Decide the authority. To deploy via Intune, move the **Client apps** workload to **Intune** (or **Pilot Intune** scoped to the pilot collection). If you keep apps in ConfigMgr, deploy the same package as a ConfigMgr Application with equivalent detection / supersedence.

R7. Entra device groups for deployment rings

Description: Microsoft Entra **device** groups for the population and the rings: `EPIC – Clinical Workstations` (all in-scope clinical endpoints), `Epic Hyperdrive – Pilot Ring`, and `Epic Hyperdrive – Broad Ring`. **Why required:** Ring-based rollout is how a bad client build is contained to a few validated devices. Assignments target these named groups. **How to verify:** Entra → **Groups** → **All groups**; the three groups exist with intended membership. **How to implement if missing:** Create them in Entra (or run `EPICCLIENT-Set-DeploymentRings.ps1`), which creates `Epic Hyperdrive – Pilot Ring` and `Epic Hyperdrive – Broad Ring` and can wire ring assignments via Microsoft Graph.

R8. Detection / version strategy

Description: A decided, reliable way for Intune to detect "the target Epic version is installed" — a **file version** of in , a **registry** uninstall key/version, or an **MSI product code** — matching how actually registers. **Why required:** The detection rule is what tells Intune the app is installed and at the right version; a wrong rule causes endless re-installs or false "installed" states. **How to verify:** On the test device after a manual install, inspect the file version of and the registry uninstall key; run `EPICCLIENT-Discovery.ps1` to read the installed version. Confirm those values match what your detection rule will assert. **How to implement if missing:** Choose the most stable signal (file version is usually best for the Epic client), and record the exact detection values for the Implementation Guide.

R9. A test clinical device (not in patient care)

Description: A representative, enrolled, non-production clinical device (ideally one fixed workstation and one WOW) in `Epic Hyperdrive – Pilot Ring`. **Why required:** Packaging, detection, launch, and Imprivata-tap validation must be proven before any device in care receives the client. **How to verify:** The device is enrolled, compliant, and a member of `Epic Hyperdrive – Pilot Ring`. **How to implement if missing:** Set aside the device and add it to the pilot ring group. See global requirement G7.

R10. Approved clinical change window and rollback plan

Description: A change-control approval, an agreed **clinical maintenance window** for the broad deployment, and a documented **rollback** (revert to the previously validated package/version). **Why required:** Deploying or reverting the EHR client touches patient-care devices; it must be change-managed, windowed, and reversible. **How to verify:** A CAB-approved change ticket exists with window and back-out plan, naming the previously validated package to revert to. **How to implement if missing:** Route through your CAB; keep the prior validated `.intunewin` and its detection values for rollback. See global requirements G4, G5, G8.

R11. Imprivata present (only if integrating SSO / badge-tap)

Description: If your clinical endpoints use **Imprivata** for single sign-on / badge-tap into Epic, confirm the Imprivata agent is installed and healthy on the test device **before** changing the Epic client. **Why required:** A new Epic client version can change launch behavior; you must confirm tap-to-launch into Epic still works after the change. (Imprivata configuration itself is a separate solution; here it is a **validation touchpoint** only.) **How to verify:** On the test device, confirm the Imprivata agent service is running and a badge tap launches into Epic prior to the change; re-test after (Testing §, Test 6). **How to implement if missing:** Engage the separate Imprivata solution / your Imprivata administrators. If you do **not** use Imprivata, this requirement is N/A — skip the related test.

Requirements checklist (use before packaging)

- **R1** Intune app-management rights (Intune Administrator / Application Manager)
- **R2** Win32 Content Prep Tool (`IntuneWinAppUtil.exe`) available
- **R3** Epic installer + silent switches + version + install dir + exe + uninstall cmd from Epic TS/UserWeb
- **R4** Supported, patched Windows 10/11 (x64)
- **R5** WebView2 runtime present (Hyperdrive) — `EPICCLIENT-Validation.ps1`

- **R6** Co-management **Client apps** workload authority decided (if co-managed)
- **R7** Entra groups `EPIC – Clinical Workstations`, `Epic Hyperdrive – Pilot Ring`, `Epic Hyperdrive – Broad Ring`
- **R8** Detection / version strategy chosen and verified — `EPICCLIENT-Discovery.ps1`
- **R9** Test clinical device (fixed + WOW) in the pilot ring
- **R10** CAB-approved change window + rollback to prior validated package
- **R11** Imprivata agent healthy on test device (only if integrating badge-tap)

3. Implementation Guide — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune

Follow these steps **in order**. Each step must be complete before the next. Unless stated otherwise, all navigation is in the **Microsoft Intune admin center** (`https://intune.microsoft.com`).

> ***Model:*** *Intune-managed, Microsoft Entra–joined clinical Windows endpoints. We wrap the Epic client as a* ***Win32 app*** *(`.intunewin`), pin it to a version with a* ***detection rule****, gate it with a* ***requirement rule****, assign it to* `Epic Hyperdrive – Pilot Ring` *first,* ***validate****, then expand to* `Epic Hyperdrive – Broad Ring`*. Upgrades are delivered via* ***supersedence****; old versions are* ***retired*** *with a clean uninstall. Phases:* ***Assess*** *→* ***Package*** *→* ***Pilot*** *→* ***Validate*** *→* ***Broad Deploy*** *→* ***Lifecycle****, exactly like the standard deployment model.*
>
> ***Epic-specific values:*** *wherever you see , , , , , , substitute the* ***real*** *values from your* ***Epic build team / Epic UserWeb*** *(Requirements §R3). Do not invent them.*

Phase A — Assess

Step 1 — Confirm requirements and decide Hyperdrive vs. Hyperspace

1. Work through the **Requirements Guide** (§R1–R11) and the **global-requirements-healthcare-solutions** checklist. Do not proceed past any unmet item.
1. Confirm with your Epic build team **which client** you are deploying — **Hyperdrive** (current, WebView2-based) and/or **Hyperspace** (legacy) — and the exact your Epic environment expects.
1. If deploying **Hyperdrive**, confirm the **WebView2 runtime** plan (Requirements §R5) — you will wire it as a dependency in Step 6.
1. If devices are **co-managed**, confirm the **Client apps** workload authority (Requirements §R6).

Step 2 — Manually install once on the test device (prove the silent command)

1. On the **test device** (in `Epic Hyperdrive – Pilot Ring`, not in patient care), run by hand from an elevated prompt.
1. Confirm it installs **unattended** (no prompts) and that appears in

.

1. Run `EPICCLIENT-Discovery.ps1` and note the **installed version**, install path, and registry uninstall key — these are your **detection** values (Requirements §R8).

1. Launch the client, confirm it loads and points at the correct Epic environment, then (if testing a clean package) uninstall with to return the device to a known state.

Phase B — Package

Step 3 — Build the `.intunewin` package

1. On your packaging workstation, create a clean **source folder** (e.g. `C:\EpicPkg\Source`) and place

and any required supporting files inside it.

1. Run the Win32 Content Prep Tool:

```
IntuneWinAppUtil.exe -c "C:\EpicPkg\Source" -s "<EPIC_INSTALLER>" -o "C:\EpicPkg\Output"
```

1. Confirm `C:\EpicPkg\Output\.intunewin` is produced.

Step 4 — Create the Win32 app in Intune

1. Go to **Apps** → **Windows** → **Add**. For **App type**, select **Windows app (Win32)** → **Select**.

2. On **App information**:

• **Select app package file** → upload …`\.intunewin` → **OK**.

• **Name:** `Epic Hyperdrive –` (for the legacy client use

`Epic Hyperspace –`). Use this exact name in all later steps.

• **Publisher:** your organization. Fill **Description** (note the Epic environment and version).

• Click **Next**.

Step 5 — Program (install / uninstall commands)

1. On **Program**:

• **Install command:** (the exact silent command proven in

Step 2).

• **Uninstall command:** (silent uninstall from your Epic build team).

• **Install behavior: System** (clinical endpoints are shared / multi-user).

• **Device restart behavior: No specific action** (do not force reboots on clinical devices;

control restarts through the maintenance window).

1. Click **Next**.

Step 6 — Requirements rule (and WebView2 dependency for Hyperdrive)

1. On **Requirements**:

• **Operating system architecture: 64-bit**.

• **Minimum operating system:** the lowest supported build per Requirements §R4 (e.g. Windows 10 21H2 / Windows 11).

• Add any additional requirement your build team specifies (disk space, etc.).

1. Click **Next**.
2. **WebView2 dependency (Hyperdrive only):** if you deployed the **WebView2 runtime** as its own Win32 app (Requirements §R5), set it as a dependency after the app is created — open `Epic Hyperdrive –` → **Dependencies** → **+ Add**, select the WebView2 runtime app, set **Automatically install** = **Yes**, so WebView2 installs first. (Dependencies are configured on the saved app; you can complete the wizard first and add this immediately after Step 9.)

Step 7 — Detection rule (pin the version)

1. On **Detection rules**, set **Rules format** to **Manually configure detection rules** → **+ Add**.
2. Choose the rule type that matches how registers (Requirements §R8):

• **File** (recommended): **Path** = , **File** = , **Detection method** = **String (version)**, **Operator** = **Greater than or equal to**, **Value** = . Name it conceptually `Epic Client – Version Detection`.

• **or MSI** (product code) / **Registry** (uninstall key + DisplayVersion) if that is how it registers.

1. Click **OK** → **Next**.

Step 8 — (Optional) Detection / requirement scripts; Return codes; Scope tags

1. Leave **default return codes** unless your installer uses non-standard codes.
2. Add **Scope tags** if your org uses them, else **Next**.

Step 9 — Create the app (assign in Phase C)

1. On **Assignments**, leave assignments empty for now → **Next**.
2. On **Review + create**, confirm everything → **Create**.
3. **Now wire the WebView2 dependency** (Step 6.3) if deploying Hyperdrive.

Phase C — Pilot

Step 10 — Ensure the ring groups exist

1. In `https://entra.microsoft.com` → **Groups** → **All groups**, confirm `EPIC – Clinical Workstations`, `Epic Hyperdrive – Pilot Ring`, and `Epic Hyperdrive – Broad Ring` exist (Requirements §R7).

1. If missing, create them (**New group** → **Security**) or run

`EPICCLIENT-Set-DeploymentRings.ps1` to create the pilot/broad ring groups and (optionally) wire the assignment via Graph.

1. Add your **test device(s)** to `Epic Hyperdrive – Pilot Ring`.

Step 11 — Assign the app to the Pilot Ring

1. Open **Apps** → **Windows** → `Epic Hyperdrive –` → **Properties** → **Assignments** → **Edit**.
2. Under **Required**, click **+ Add group** → select `Epic Hyperdrive – Pilot Ring` → **Select**.
3. Expand the assignment's **Settings** and set an **Available** time aligned to your maintenance window if desired (for **Required**, the **End user notifications** and restart grace can be set here). Click **Review + save** → **Save**.

Step 12 — Sync and install on a pilot device

1. On a pilot device, force a sync: **Settings → Accounts → Access work or school → (account) → Info → Sync**.

1. In Intune, watch **Apps** → **Windows** → `Epic Hyperdrive –` → **Device install status** until the pilot device shows **Installed**.

Phase D — Validate

Step 13 — Validate the pilot device

1. On the pilot device, run `EPICCLIENT-Validation.ps1` — confirms the **target version** installed, install state, and (Hyperdrive) the **WebView2 runtime** present → expect overall **PASS**.

1. Launch the Epic client; confirm it **loads** and connects to the correct Epic environment.
2. If integrating Imprivata (Requirements §R11), perform a **badge tap** and confirm tap-to-launch into Epic still works.

1. Complete the **Testing Guide (Section 5)**. **Do not expand** until every pilot test passes.
2. Run `EPICCLIENT-Report.ps1` to capture the pilot's version spread as a baseline.

Phase E — Broad Deploy

Step 14 — Assign to the Broad Ring (in the change window)

1. Confirm the CAB-approved change window is open (Requirements §R10).
2. Open **Apps** → **Windows** → `Epic Hyperdrive –` → **Properties** → **Assignments** → **Edit**.
3. Under **Required**, **+ Add group** → `Epic Hyperdrive – Broad Ring` → **Select** → **Review + save** → **Save**.

1. Monitor **Device install status** and re-run `EPICCLIENT-Report.ps1` to watch the version converge across the fleet. Re-run `EPICCLIENT-Validation.ps1` on a sample of broad devices.

Phase F — Lifecycle (update / supersede / retire)

Step 15 — Deliver an Epic client upgrade via supersedence

When Epic moves your environment to a new :

1. Repeat **Steps 2–9** to build and create a **new** Win32 app

`Epic Hyperdrive –` with its own version-pinned detection rule.

1. On the **new** app, go to **Supersedence** → **+ Add**.
2. Select the **previous** app `Epic Hyperdrive –` as the app to supersede.
3. Set **Uninstall previous version** = **Yes** (Intune removes the old client, then installs the new

one — the clean upgrade path). Name this relationship conceptually `Epic Hyperdrive – Supersede` →. Click **Add** → **Save**.

1. **Assign the new app to** `Epic Hyperdrive – Pilot Ring` **first** (Step 11), validate (Phase D), then

expand to `Epic Hyperdrive – Broad Ring` (Phase E). Supersedence respects ring assignments — pilot upgrades before broad.

1. Re-run `EPICCLIENT-Report.ps1` until the fleet shows only .

> ***Note:*** *supersedence requires the* ***new*** *app to be assigned to devices for the replacement to occur. Assigning only the new app (not re-touching the old) and confirming* ***Uninstall previous version = Yes*** *is what produces an in-place upgrade rather than two coexisting versions.*

Step 16 — Retire an old client version cleanly

To remove a decommissioned version that is **not** being replaced by a superseding app:

1. Open the old app `Epic Hyperdrive –` → **Properties** → **Assignments** → **Edit**.
2. Remove its **Required** assignments, and under **Uninstall** add the target group (e.g.

`EPIC – Clinical Workstations` or the specific ring) → **Save**. Intune runs the app's **Uninstall command** () on those devices.

1. After devices report the uninstall and `EPICCLIENT-Discovery.ps1` shows the version gone,

you may delete the old Win32 app from **Apps** → **Windows** (keep a copy of its `.intunewin` and detection values for rollback per Requirements §R10 until the new version is fully proven).

1. Spot-check with `EPICCLIENT-Validation.ps1` that no leftover old version remains.

Order of operations (summary)

```
Assess + prove silent install → Build .intunewin → Create Win32 app (program, requirement, WebView2 dependency, version detection) → Ring groups → Assign Pilot Ring → Validate (version, launch, Imprivata tap) → Assign Broad Ring (in change window) → Lifecycle: supersede for upgrades (uninstall previous = Yes, pilot-then-broad) / retire via Uninstall assignment
```

5. Testing Guide — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune

Run these validations after the pilot is deployed (Implementation Phase D) and again after the broad deployment and after every lifecycle change (upgrade / retire). Each test states **what to do** and the **expected (PASS) result**. **Do not expand the rollout until every pilot test passes.**

> *Run on a device in* `Epic Hyperdrive – Pilot Ring` *first. Use the scripts where named; capture results in your change log for the clinical change record.*

Test 1 — The Epic client installed

Do: On a pilot device run `EPICCLIENT-Discovery.ps1`, or check for . **PASS:** The Epic client is present in ; Discovery reports an installed version. **FAIL:** Not installed — see Troubleshooting §6 (install failures) and `EPICCLIENT-Remediation.ps1`.

Test 2 — The installed version is the target version

Do: Run `EPICCLIENT-Validation.ps1` (compares the installed version to the target), or read the file version of . **PASS:** Installed version **equals** (the value pinned in the detection rule). **FAIL:** Version mismatch — detection rule or supersedence issue (Troubleshooting §6, version drift).

Test 3 — Intune reports the app Installed (no errors)

Do: Intune → **Apps** → **Windows** → `Epic Hyperdrive –` → **Device install status**. **PASS:** Pilot devices show **Installed**; no **Failed** or detection-mismatch states. **FAIL:** Failed / "Not installed" despite the binary present → detection-rule mismatch (Troubleshooting §6).

Test 4 — The client launches and loads

Do: On the pilot device, launch the Epic client (Hyperdrive or Hyperspace) as a clinical user. **PASS:** The client opens, renders fully, and connects to the **correct Epic environment**. For **Hyperdrive**, the WebView2 surface renders (no blank / white window). **FAIL:** Crash, blank window, or wrong environment → Troubleshooting §6 (WebView2 / launch issues).

Test 5 — WebView2 runtime present (Hyperdrive)

Do: Run `EPICCLIENT-Validation.ps1` (checks the WebView2 runtime). **PASS:** The **Microsoft Edge WebView2 Runtime** is present and reported. **FAIL:** Missing → deploy/repair WebView2 (Requirements §R5, Implementation Step 6). *(N/A for Hyperspace-only deployments.)*

Test 6 — Imprivata badge-tap into Epic still works (if integrating)

Do: On the pilot device, perform a **badge tap** and confirm tap-to-launch / SSO into the Epic client. **PASS:** The tap launches and authenticates into Epic as before the client change. **FAIL:** Tap no longer launches Epic → confirm the Imprivata agent is healthy and engage the separate Imprivata solution / administrators (this solution does not configure Imprivata). *(N/A if you do not use Imprivata — Requirements §R11.)*

Test 7 — No leftover old version after an upgrade / retire

Do: After a supersedence upgrade (Step 15) or a retire (Step 16), run `EPICCLIENT-Discovery.ps1` / `EPICCLIENT-Validation.ps1`. **PASS:** Only the **intended** version is present (or, for a retire, **no** Epic client remains); no second/old version coexists in `Apps & features` or . **FAIL:** Old version still present → Troubleshooting §6 (supersedence not upgrading / retire leftovers).

Test 8 — Ring rollout is healthy

Do: Run `EPICCLIENT-Report.ps1` and review Intune **Device install status** per ring. **PASS:** `Epic Hyperdrive – Pilot Ring` is fully Installed at the target version **before** broad; after broad, `Epic Hyperdrive – Broad Ring` converges to the target version with an acceptable failure rate. **FAIL:** Stuck installs / mixed versions in a ring → Troubleshooting §6 (ring / assignment problems).

Test 9 — Device compliance and clinical function intact

Do: Intune → **Devices** → **Compliance** for the pilot/broad devices; confirm clinical peripherals relied on at launch (badge reader, etc.) still function. **PASS:** Devices remain **Compliant**; no new failures introduced by the client change. **FAIL:** New compliance failures or broken clinical function → roll back per Requirements §R10.

Acceptance gate

Gate	Requirement
Pilot pass (required before broad deploy)	Tests **1, 2, 3, 4, 8, 9** pass for **100%** of `Epic Hyperdrive – Pilot Ring`; **Test 5** passes for Hyperdrive; **Test 6** passes if Imprivata is integrated
Broad pass	Tests **1, 2, 3, 4, 8** pass across `Epic Hyperdrive – Broad Ring` at an agreed success threshold; **Test 9** shows no new compliance failures
Lifecycle pass (after upgrade / retire)	**Test 7** passes (no leftover old version), plus Tests 2, 4, 8 for the new state

Record results in the clinical change log. If any test fails, use the **Troubleshooting Guide (Section 6)** and `EPICCLIENT-Remediation.ps1` before proceeding, and trigger the rollback plan if the failure affects patient-care devices.

6. Troubleshooting Guide — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune

Work top-down: confirm the requirement, reproduce on the **test device** first, and only then act on patient-care devices. Where a script helps, it is named. For Epic-installer specifics (switches, version

strings, environment config), confirm with your **Epic build team / Epic UserWeb** — this guide covers the **Intune / ConfigMgr** mechanics.

A. Install fails or never starts

Symptom: Intune → **Apps** → **Windows** → `Epic Hyperdrive –` → **Device install status** shows **Failed**, or the install never runs.

1. **Silent command not actually silent.** Re-run by hand on the

test device (Implementation Step 2). If it prompts, the Intune **Install command** is wrong — get the correct silent switches from your Epic build team and update **Program** → **Install command**.

1. **Requirement rule excludes the device.** Check **Properties** → **Requirements** — wrong minimum OS /

architecture will leave the app **Not applicable**. Confirm the device meets it (`winver`).

1. **WebView2 dependency failing (Hyperdrive).** If the WebView2 dependency app is failing, the Epic

app won't proceed. Check the WebView2 app's own install status; fix it first (Section D).

1. **Install behavior wrong.** Shared clinical devices need **Install behavior = System**. A

per-user install on a shared device misbehaves — set it to **System**.

1. **Non-standard return codes.** If returns a code Intune treats as failure,

map it under **Program** → **Return codes** (e.g. add the installer's success/soft-reboot codes).

1. **Nudge the device:** run `EPICCLIENT-Remediation.ps1 -AppName "Epic Hyperdrive - " -Apply`

to force an MDM sync, wait, then re-check.

B. Detection mismatch (installs but Intune says "Not installed", or re-installs in a loop)

Symptom: The Epic client is clearly on the device, but Intune reports **Not installed** (or keeps reinstalling).

1. **Detection value doesn't match reality.** On the test device run `EPICCLIENT-Discovery.ps1` and

compare the **actual** version / path / registry key to the **detection rule** (Implementation Step 7). They must match exactly.

1. **Wrong detection type.** If the installer registers an MSI product code, a **File version** rule

on the wrong path won't detect it (and vice-versa). Switch the rule type to match how actually registers (Requirements §R8).

1. **Operator too strict/loose.** A **String (version) – Greater than or equal to** rule against

is usually right. An **Equals** on a version that the installer formats slightly differently will read as "not installed."

1. **Path with environment specifics.** Ensure / in the rule are the

real installed path/name, not a guess.

C. Version drift (devices stuck on an old version)

Symptom: `EPICCLIENT-Report.ps1` shows a mix of versions; some devices never moved to .

1. **Device not in the assigned ring.** Confirm the device is a member of `Epic Hyperdrive – Pilot Ring / Epic Hyperdrive – Broad Ring (whichever the new app is assigned to). Add it if missing (or run EPICCLIENT-Set-DeploymentRings.ps1`).
1. **Superseding app not assigned.** Supersedence only acts where the **new** app is assigned — see Section E.
1. **Device not syncing.** Force a sync (**Access work or school** → **Sync**) or run `EPICCLIENT-Remediation.ps1 … -Apply`.
1. **Stale install failure** on those devices — drill into **Device install status** for the error and apply Section A.

D. Hyperdrive won't launch (blank/white window, crash) — WebView2

Symptom: Hyperdrive installs but shows a blank/white window, fails to render, or crashes at launch.

1. **WebView2 runtime missing.** This is the most common cause. Run `EPICCLIENT-Validation.ps1`; if **WebView2 runtime present = FAIL**, deploy/repair the **Evergreen WebView2 Runtime** and set it as a **dependency** (auto-install = Yes) of the Hyperdrive app (Requirements §R5, Implementation Step 6).
1. **WebView2 present but broken.** Repair/reinstall the WebView2 runtime app via Intune, then relaunch.
2. **Wrong Epic environment / config.** If it launches but connects to the wrong environment, the client configuration is off — confirm the environment/connection values with your Epic build team (out of scope for packaging, but a common "looks broken" report).
1. **Hyperspace, not Hyperdrive.** If you are deploying legacy **Hyperspace**, WebView2 is N/A — don't chase a WebView2 fix for a Hyperspace launch issue.

E. Supersedence not upgrading (old version stays)

Symptom: You created a new app and a supersedence relationship, but devices keep the old version.

1. **New app not assigned to the devices.** Supersedence requires the **new** app to be **assigned** (Required) to the ring (Implementation Step 15.5). Assign `Epic Hyperdrive –` to the pilot ring first, then broad.
1. **"Uninstall previous version" not set.** On the new app → **Supersedence**, confirm the relationship to `Epic Hyperdrive –` has **Uninstall previous version = Yes** for a clean replace.
1. **Old app's detection still satisfied / old app still required.** If the **old** app is still

assigned **Required** to the same devices, the two assignments fight. Remove the old app's Required assignment once supersedence is in place.

1. **Not synced yet.** Allow time and force a sync; supersedence evaluation isn't instant.
2. **Detection on the new app wrong.** If the new app's version-pinned detection is wrong, Intune thinks the upgrade failed — re-check per Section B.

F. Retire leftovers (old client remains after retire)

Symptom: After retiring a version (Implementation Step 16), the old client is still present.

1. **Uninstall command wrong.** The app's **Program** → **Uninstall command** () must silently remove the client. Test it by hand on the test device; correct it with your Epic build team's switches.
1. **Uninstall assignment missing the device.** Confirm you added the device's group to the app's **Uninstall** assignment, and removed any **Required** assignment that would reinstall it.
1. **Verify and finish.** Run `EPICCLIENT-Discovery.ps1` / `EPICCLIENT-Validation.ps1`; only delete the old Win32 app after devices report the uninstall (keep the prior `.intunewin` for rollback per Requirements §R10).

G. Ring / assignment problems

Symptom: The wrong devices got the app, or a ring is mixed/stuck.

1. **Group membership.** Confirm `EPIC – Clinical Workstations`, `Epic Hyperdrive – Pilot Ring`, and `Epic Hyperdrive – Broad Ring` contain the intended **devices** (these are device groups). Fix membership in Entra or via `EPICCLIENT-Set-DeploymentRings.ps1`.
1. **Assigned too broadly too soon.** If broad got it before pilot was validated, remove the broad **Required** assignment, re-validate pilot, then re-add broad in the change window.
1. **Overlapping/contradictory assignments.** Avoid assigning both **Required** and **Uninstall** for the same app to overlapping groups — they conflict. Keep one intent per group.
1. **Co-management authority.** If devices are co-managed and the **Client apps** workload points at ConfigMgr, Intune app assignments won't apply — set the workload authority correctly (Requirements §R6).

H. Imprivata badge-tap no longer launches Epic after a client change

Symptom: Tap-to-launch / SSO into Epic stopped working after deploying/upgrading the client.

1. **Confirm Imprivata agent health** on the device (service running). Imprivata configuration is a **separate solution** — this solution only validates the tap.
1. **Launch target changed.** A new client version can change the launch executable/path Imprivata

targets; engage your Imprivata administrators / the Imprivata solution to update the profile.

1. **Re-test** with Testing §6 once corrected. If you don't use Imprivata, this is N/A.

Escalation / rollback

If a failure affects **patient-care** devices and a quick fix isn't in hand, **invoke the rollback plan** (Requirements §R10): re-assign the previously validated package/version (or remove the new assignment), confirm with `EPICCLIENT-Validation.ps1`, and record the back-out in the clinical change log. For Epic-installer behavior beyond packaging (switches, version strings, environment config), escalate to your **Epic build team / Epic Technical Services**.

7. Best Practices — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune

These practices keep the Epic client deployment safe, version-coherent with Epic, and clean over its lifecycle on shared clinical endpoints.

Source of truth: Epic owns the installer, you own the delivery

• **Always** take the installer, silent switches, version string, and environment configuration from your **Epic build team / Epic UserWeb** — never guess Epic-internal flags or version numbers.

• Re-confirm these values **each Epic upgrade**; they can change between Epic versions.

• Keep a small internal record (not in this package) mapping each to the Epic environment, installer, and switches you packaged.

Pin the version with detection — and keep detection honest

• Use a **version-pinned detection rule** (file version of is usually the most stable).

• After every package, **verify the detection value against a real install** on the test device (`EPICCLIENT-Discovery.ps1`) before assigning — a wrong rule causes endless re-installs or false "installed" states.

• Prefer **file version** over a registry DisplayVersion if the installer's registry value is inconsistent across builds.

Always pilot before broad — and let supersedence respect rings

• Every release and every **upgrade** goes to `Epic Hyperdrive – Pilot Ring` and is fully validated (Testing §) **before** `Epic Hyperdrive – Broad Ring`.

• For upgrades, assign the **new** superseding app to the pilot ring first; supersedence respects assignment scope, so the broad fleet only upgrades once pilot is proven.

• Keep the pilot ring **representative** — include at least one fixed clinical workstation and one **WOW**, since form factors behave differently (battery, dock, network).

Treat the WebView2 runtime as a first-class dependency (Hyperdrive)

• Deploy the **Evergreen WebView2 Runtime** as its own app and set it as a **dependency** (auto-install = Yes) of the Hyperdrive app, so it is present **before** Hyperdrive installs/launches.

• Do **not** assume Windows 11 already has WebView2 — verify with `EPICCLIENT-Validation.ps1`.

Upgrade cleanly with supersedence; retire deliberately

• For upgrades, use **supersedence with "Uninstall previous version = Yes"** so the old client is removed before the new one installs — this prevents multiple Epic versions accreting on shared devices.

• For decommissioned versions not being superseded, retire via an **Uninstall assignment**, then verify nothing is left behind (`EPICCLIENT-Discovery.ps1`).

• Keep the **previous validated** `.intunewin` and its detection values until the new version is fully proven, so rollback is one assignment away.

Respect the clinical environment

• **Install behavior = System**, **device restart behavior = No specific action** — never force a reboot on a shared clinical device; control restarts through the approved **maintenance window**.

• Align assignment **availability** to the maintenance window; communicate changes to clinical operations in advance.

• Every deploy / upgrade / retire is a **change-controlled** action with an approved window and a documented **rollback** (global requirements G4, G5, G8).

Co-management clarity

• If devices are co-managed, decide **one** authority for **client apps** (Intune **or** ConfigMgr) and stick to it, so the Epic client is never deployed twice from two systems.

Protect ePHI in operations

• The tools in this package are **read-only over device/app state** and must **never** capture patient data. Keep logs and CSV reports limited to device, version, and install-state fields.

• Restrict who can run deployment/uninstall actions; retain change records per your compliance program.

Operate on a cadence

• Run `EPICCLIENT-Report.ps1` on a schedule to watch for **version drift** (devices that fell behind) and to confirm each ring is converged.

• After any change, sample devices with `EPICCLIENT-Validation.ps1`; use `EPICCLIENT-Remediation.ps1` (`-WhatIf` first, then `-Apply`) for one-off recoveries rather than re-broadcasting to a whole ring.

8. Deliverables Summary — Epic Hyperdrive & Hyperspace Client Deployment & Lifecycle Management with Microsoft Intune

This solution is delivered as a complete, click-to-implement package: implementation documentation, an operational PowerShell toolkit, and a hero image.

Documentation

Document	Purpose
executive-summary.md	Business-level overview, value, and approach for decision-makers.
01-solution-overview.md	What the solution delivers, devices in scope, scope boundaries, and how to use the package.
02-requirements.md	Epic-deployment-specific requirements (R1–R11), each with Description / Why / Verify / Implement, plus a checklist.
global-requirements-healthcare-solutions.md	Category-wide prerequisites: tenant + roles, Windows licensing, HIPAA/HITRUST change control, clinical maintenance windows, network, test device, rollback.
03-implementation.md	Ordered, full-navigation steps: package the Win32 `.intunewin`, version-pinned detection, ring groups, pilot assign, validate, broad deploy, then lifecycle (supersede / retire).
05-testing.md	Numbered tests with PASS/FAIL gates and the acceptance gate that must pass before expansion.
07-best-practices.md	Operating practices for version pinning, ring discipline, WebView2, supersedence/retire, and clinical-environment safety.
06-troubleshooting.md	Fixes for install failures, detection mismatch, version drift, Hyperdrive/WebView2 launch issues, supersedence not upgrading, retire leftovers, and ring/assignment problems.

PowerShell toolkit (`scripts\`)

Script	What it does
EPICCLIENT-Discovery.ps1	Inventories the installed Epic client version and basic health across managed devices. Read-only.
EPICCLIENT-Validation.ps1	Verifies a device has the **target version**, the app install state, and launch prerequisites (WebView2 for Hyperdrive); emits PASS/FAIL and an overall verdict.
EPICCLIENT-Report.ps1	Produces a fleet rollout / version report (per-ring counts, version spread) to **CSV**.
EPICCLIENT-Remediation.ps1	Safely re-triggers install or uninstall of a **named** Win32 app for a device (MDM sync / install-state nudge). Read-only / `-WhatIf` by default; `-Apply` to act.
EPICCLIENT-Set-DeploymentRings.ps1	Creates the `Epic Hyperdrive – Pilot Ring` and `Epic Hyperdrive – Broad Ring` Entra device groups and (optionally) wires ring assignments via Microsoft Graph. Supports `-WhatIf`.

Each script carries a `.SYNOPSIS` plus **PURPOSE / HOW TO RUN / EXPECTED RESULTS** in its header.

Visual asset

Asset	Purpose
EPICCLIENT-hero.png	Hero image for the solution listing.

Named objects you create (reused verbatim across the docs and scripts)

- **Win32 app:** `Epic Hyperdrive –` (legacy: `Epic Hyperspace – `)
- **Detection rule:** `Epic Client – Version Detection`
- **Supersedence relationship:** `Epic Hyperdrive – Supersede` →
- **Entra device groups / rings:** `EPIC – Clinical Workstations`, `Epic Hyperdrive – Pilot Ring`,

`Epic Hyperdrive – Broad Ring`

*Epic-specific values (, , , ,) come from your **Epic build team / Epic UserWeb** and are substituted consistently throughout.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

EPICCLIENT-Discovery.ps1

Purpose: Reports, on the device it runs on, which Epic client is installed (Hyperdrive and/or Hyperspace), its version, install path, and basic health (executable present, WebView2 runtime present for Hyperdrive). Use during Assess (Implementation Step 2) to capture the detection values, and at any time to inventory a device. Read-only.

How to run: On the device, in a normal PowerShell window: powershell -ExecutionPolicy Bypass -File .\EPICCLIENT-Discovery.ps1 Optionally point it at the known install path / executable from your Epic build team: .\EPICCLIENT-Discovery.ps1 -InstallDir "C:\Program Files\Epic" -Exe "Hyperdrive.exe"

EPICCLIENT-Remediation.ps1

Purpose: Recovers a single device whose Epic client deployment is stuck: it forces an Intune MDM sync (so the assigned Win32 app and its detection re-evaluate and install/uninstall actions reach the device) and reports the current Epic client state. With -Apply it triggers the sync; by default it runs read-only / -WhatIf so you can preview. It does NOT silently push installers on its own — the actual install/uninstall is performed by the Intune-assigned named app ("Epic Hyperdrive - "); this script nudges the device to act on that assignment.

How to run: On the affected device. Preview first (no changes): powershell -ExecutionPolicy Bypass -File .\EPICCLIENT-Remediation.ps1 -AppName "Epic Hyperdrive - " Then act (triggers an MDM sync; run elevated): .\EPICCLIENT-Remediation.ps1 -AppName "Epic Hyperdrive - " -Apply

EPICCLIENT-Report.ps1

Purpose: Builds a tenant-wide rollout report for the Epic client Win32 app: per deployment ring how many devices have Installed / Failed / Pending, and the version spread across the fleet, using Microsoft Graph (Intune) data. Use it to confirm a ring is converged before expanding, and to catch version drift over time. Read-only.

How to run: Requires the Microsoft.Graph PowerShell SDK and an operator with Intune read rights. Install-Module Microsoft.Graph -Scope CurrentUser # one time powershell -ExecutionPolicy Bypass -File .\EPICCLIENT-Report.ps1 `-AppName "Epic Hyperdrive - "` -PilotGroup "Epic Hyperdrive - Pilot Ring" -BroadGroup "Epic Hyperdrive - Broad Ring" ` -OutCsv .\EpicClient-Rollout.csv

EPICCLIENT-Set-DeploymentRings.ps1

Purpose: Provisions the named deployment-ring objects this solution uses: "Epic Hyperdrive - Pilot Ring" and "Epic Hyperdrive - Broad Ring" (Microsoft Entra security groups, membershipType device-assigned), and — if you pass -AppName — wires the Epic client Win32 app's Required assignments to those rings via Microsoft Graph (pilot first). Supports -WhatIf so you can preview every create/assignment before it happens.

How to run: Requires the Microsoft.Graph SDK and an operator with group + Intune app write rights. Install-Module Microsoft.Graph -Scope CurrentUser # one time Preview (no changes): powershell -ExecutionPolicy Bypass -File .\EPICCLIENT-Set-DeploymentRings.ps1 -WhatIf Create the ring groups

only: .\EPICCLIENT-Set-DeploymentRings.ps1 Create groups AND assign the app to the pilot ring (Required): .\EPICCLIENT-Set-DeploymentRings.ps1 -AppName "Epic Hyperdrive - " -AssignRing Pilot

EPICCLIENT-Validation.ps1

Purpose: Validates, on a pilot/broad device, that (1) the Epic client is installed, (2) the installed version matches the TARGET version you pinned in the Intune detection rule, (3) the device is Intune-managed/Entra-joined, and (4) launch prerequisites are present (the Edge WebView2 runtime for Hyperdrive). Emits PASS/FAIL per check and an overall PROVISIONED / NOT PROVISIONED verdict. Run after Implementation Step 12 (pilot) and after each ring/lifecycle change. Read-only.

How to run: On the device, in a normal PowerShell window. Supply your Epic build team's values: powershell -ExecutionPolicy Bypass -File .\EPICCLIENT-Validation.ps1 ` -InstallDir "C:\Program Files\Epic" -Exe "Hyperdrive.exe" -TargetVersion "10.x.x" -Client Hyperdrive For the legacy client use -Client Hyperspace (WebView2 check is skipped).